Excercises Lab 4 Bonus

Hash Functions - MACs

1 Recovering Credentials Using Hashcat

1.1 Problem

We are given a target SHA-256 hash originating from the concatenation of two unknown words selected from the `SecLists/500-worst-passwords` file. The login system verifies input by hashing a string of the format:

$$H(x) = \text{SHA256}(\texttt{username\_password})$$

To authenticate successfully, we must discover the correct pair (u, p) such that:

SHA256(`u_p`) = `ae69b741dbd42018bbcc2a3b7f22b3152571ddbe8b6b562665d025a455b0a36f`

1.2 Search Space Analysis

The dictionary contains 500 weak password candidates. Both the username and password are drawn from this same set:

$$|\mathcal{W}| = 500$$

Since the system constructs credentials as `word1_word2`, the total credential keyspace is:

$$N = 500^2 = 250{,}000$$

A quarter-million candidates is computationally trivial for modern hardware, making offline brute-force enumeration with Hashcat entirely feasible.

1.3 Solution Approach

Rather than brute-forcing with Python, we generate every possible candidate string and use Hashcat to test them against the target hash. This allows GPU or CPU-accelerated cracking while keeping the workflow fully reproducible inside WSL.

The cracking strategy is:

1. Download the password list.
2. Generate all combinations of `username_password`.

3. Supply the generated list to Hashcat along with the target hash.

4. Hashcat identifies the matching input whose SHA-256 digest equals the target.

1.4 Cracking Process

1.4.1 Step 1: Download the dictionary

```
(base) avduli@HP-Elvis:~$ wget https://raw.githubusercontent.com/danielmiess
ler/SecLists/master/Passwords/Common-Credentials/500-worst-passwords.txt -O
words.txt
--2025-11-28 20:29:34--  https://raw.githubusercontent.com/danielmiessler/Se
cLists/master/Passwords/Common-Credentials/500-worst-passwords.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.1
11.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.
111.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3491 (3.4K) [text/plain]
Saving to: ‘words.txt’

words.txt           100%[================>]   3.41K  --.-KB/s    in 0s

2025-11-28 20:29:34 (15.1 MB/s) - ‘words.txt’ saved [3491/3491]
```

1.4.2 Step 2: Generate all word combinations

```
(base) avduli@HP-Elvis:~$ awk 'NR==FNR{a[++n]=$0;next}{ for(i=1;i<=n;i++) pr
int $0"_"a[i] }' words.txt words.txt > combos.txt
```

1.4.3 Step 3: Confirm total combinations (Optional)

```
(base) avduli@HP-Elvis:~$ wc -l combos.txt
249001 combos.txt
```

1.4.4 Step 4: Save the target SHA256 hash

```
(base) avduli@HP-Elvis:~$ echo "ae69b741dbd42018bbcc2a3b7f22b3152571ddbe8b6b
562665d025a455b0a36f" > hash.txt
```

1.4.5 Step 5: Crack using Hashcat

It will take a while

```
(base) avduli@HP-Elvis:~$ hashcat -m 1400 hash.txt combos.txt -O
hashcat (v6.2.6) starting
```

1.4.6 Step 6: Display Credentials

```
(base) avduli@HP-Elvis:~$ hashcat -m 1400 --show hash.txt combos.txt
ae69b741dbd42018bbcc2a3b7f22b3152571ddbe8b6b562665d025a455b0a36f:spider_rock
```

1.5 Result

Hashcat successfully identifies the string

`spider_rocket`

Lets check if those are really the credentials